File Systems and Metadata

Dr Olga Angelopoulou

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Learning outcomes

- Discussion on common file systems
- Overview of FAT and NTFS
- Discussion on their relevance in a digital forensic investigation

File Systems in Forensics

1. File systems are logical structures layered over physical storage
2. They mediate:
   i. How files are created, stored, modified, and deleted
   ii. How activity is recorded and remembered
3. In digital forensics:
   i. We rarely analyse “files”
   ii. We analyse metadata, allocation, and remnants

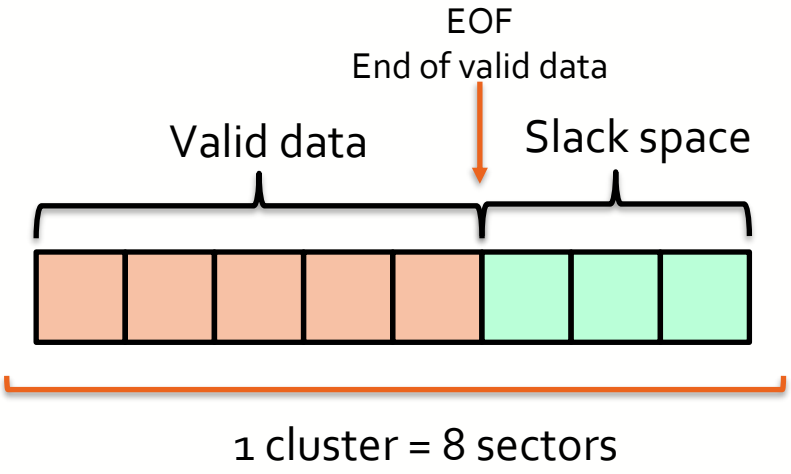
Why they matter

- File system design determines:
  - Evidence availability - What artefacts exist?
  - Evidence persistence - How long artefacts survive deletion?
  - Evidence reliability - Whether artefacts reflect user activity, system activity, or both
- Logical vs Physical Perspectives
  - Logical view
    - What the operating system presents to users - Files, folders, names, timestamps
  - Physical view
    - Sectors, clusters, allocation tables
    - Metadata structures and raw content
- Forensics operates at the physical and structural level

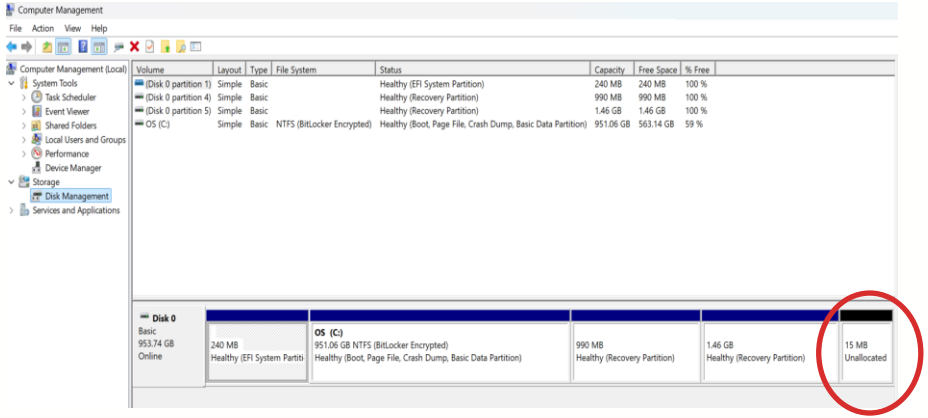
Layers of Abstraction

Slack space and Unallocated space

- Slack space
- Unused bytes within an allocated cluster
- May contain remnants of previous data



- Unallocated space
  - Disk areas no longer referenced by the file system
  - Common location for deleted content



Common File Systems and Forensic Differences

ROYAL HOLLOWAY UNIVERSITY OF LONDON

FAT / exFAT

- Simple structure, limited metadata
- Easier file recovery
- Often used on USBs, SD cards

NTFS

- Rich metadata (timestamps, permissions, logs)
- Supports advanced artefact analysis
- Common on Windows systems

ext4

- Journaling file system (Linux)
- Metadata behaviour differs from NTFS
- Artefacts often stored differently to Windows

APFS

- Apple File System
- Designed for SSDs and encryption
- Strong support for snapshots
- Recovery often more limited and complex

File Allocation Table

FAT

ROYAL HOLLOWAY UNIVERSITY OF LONDON

FAT File System

- Pointer-based allocation system
- FAT keeps very few metadata for the files
  - File fragmentation
  - All of the addressable clusters in the partition
  - Clusters marked bad
- Files are represented by:
  - Chains of clusters
  - Directory records
    - File name
    - Date/time stamps (Created, Accessed, Written)
    - Starting cluster
    - File logical size
- A directory (or folder) is a file with a unique header and a logical size of zero

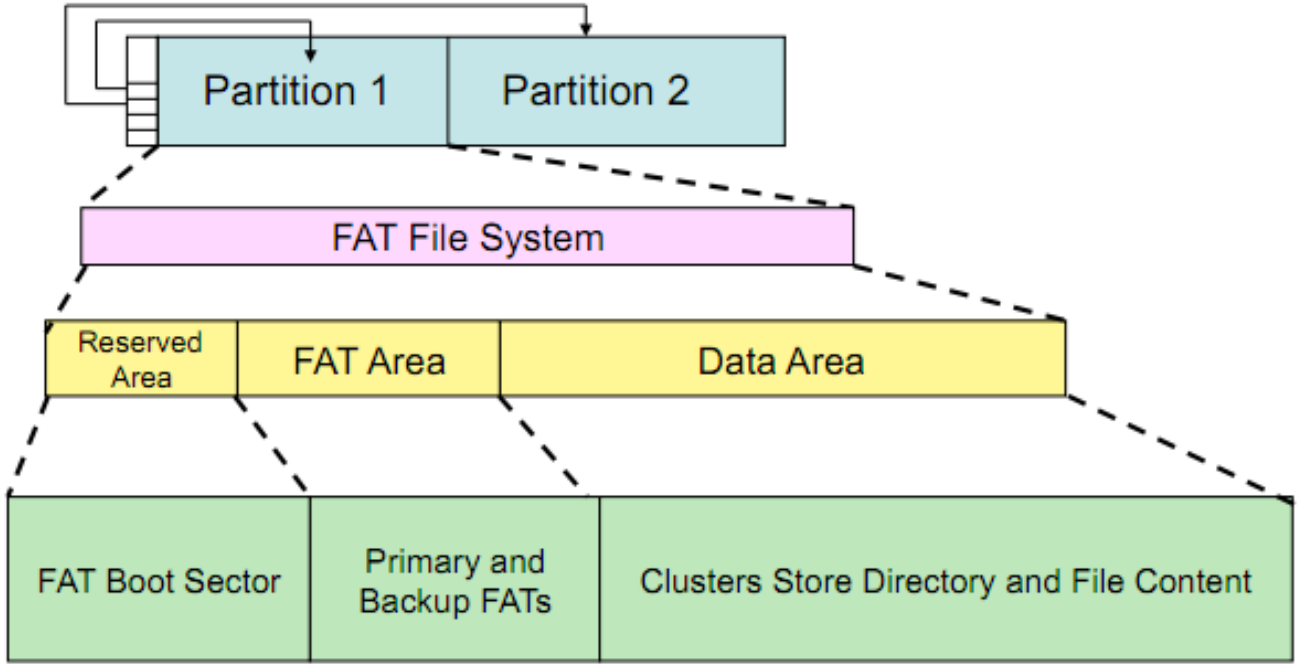
FAT

- Used by (all) Microsoft Operating Systems.
- Used by most flash memory devices
- Every file and directory is allocated a directory entry.
- Content is stored in clusters

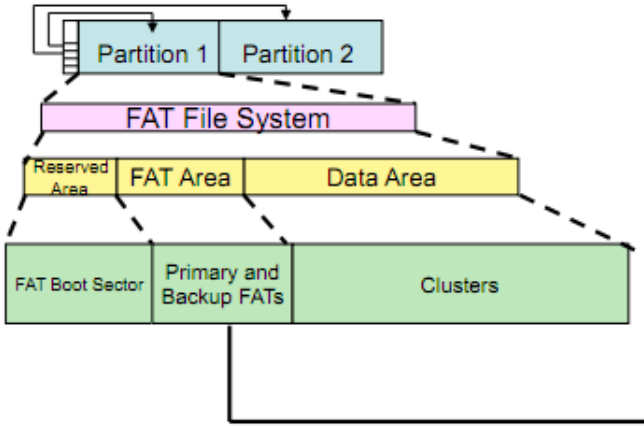
FAT12	FAT16	FAT32	VFAT	NTFS	HPFS
only recognize DOS partition sizes up to 32MB	supports partitions up to 2GB in size	FAT32 partitions can be from 32GB to 2TB in size	protected-mode version of the FAT file system (Windows 95)	2TB partitions with build-in security	better organised, faster and space saver

** HPFS allocates data to sectors instead of clusters, organized into 8mb bands. The heads don't have to return to track zero each time for access. (Netware Server, Linux Ext2)

FAT File System Layout



FAT Concepts



Cluster	FAT Entry	
000	…	
001	…	
002	000	not allocated
003	004	next cluster
004	072	next cluster
…	…	
072	FFF	end of file
…	…	

Deleting a File in Windows (FAT/FAT32)

- When a file is deleted, data is not removed from the disk
  - Directory entry is marked deleted
- FAT is updated to reflect clusters no longer in use
  - FAT chain is cleared
- New data saved to those clusters may overwrite old information

- Recovery often possible
- Timeline reconstruction is weak
- Attribution is fragile

Forensic strengths and weaknesses

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Strengths

- Predictable layout
- Manual reconstruction feasible
- Excellent for teaching fundamentals

Weaknesses

- Very limited metadata
- No journaling
- Easy to manipulate timestamps
- Poor scalability

Evidence vs Interpretation in File System Analysis

ROYAL HOLLOWAY UNIVERSITY OF LONDON

- File systems record events, not intent
- Metadata reflects:
  - User activity
  - System behaviour
  - Application processes
- Conflicting artefacts are normal, not exceptional

New Technology File System

NTFS was designed for reliability, security, and support for large storage devices.

NTFS

ROYAL HOLLOWAY UNIVERSITY OF LONDON

NTFS Scope

- Growing file sizes and system complexity
- Need for:
  - Robust metadata
  - Reliability
  - Recovery
- Shift from storage-centric to metadata-centric design

Remember: Everything is a file

- One of the most important concepts in understanding the design of NTFS is that important data are allocated to files.
  - This includes the basic file system administrative data that are typically hidden by other file systems.
  - In fact, the files that contain the administrative data can be located anywhere in the volume, like a normal file
    - Therefore, an NTFS file system does not have a specific layout like other file systems do.
    - The entire file system is considered a data area, and any sector can be allocated to a file.
    - The only consistent layout is that the first sectors of the volume contain the boot sector and boot code.

NTFS Cluster Sizes

ROYAL HOLLOWAY UNIVERSITY OF LONDON

NTFS 3.1 and later cluster sizes are established during formatting as powers of 2 of sectors (NTFS 1.x sizes differed) and internally uses only clusters but ensures operations are aligned to sector boundaries:

Volume Size			Default Cluster Size
7 MBytes	…	16 TBytes	4 KBytes
16 TBytes	…	32 TBytes	8 KBytes
32 TBytes	…	64 TBytes	16 KBytes
64 TBytes	…	128 TBytes	32 KBytes
128 TBytes	…	256 TBytes	64 KBytes

Individual MBR partitions could (owing to limitations in partition tables both for basic and dynamic disks) only deal with 2TBytes

- Workaround was to use multi-partition volumes, but GPT use is now all but exclusive

Forensic Implications

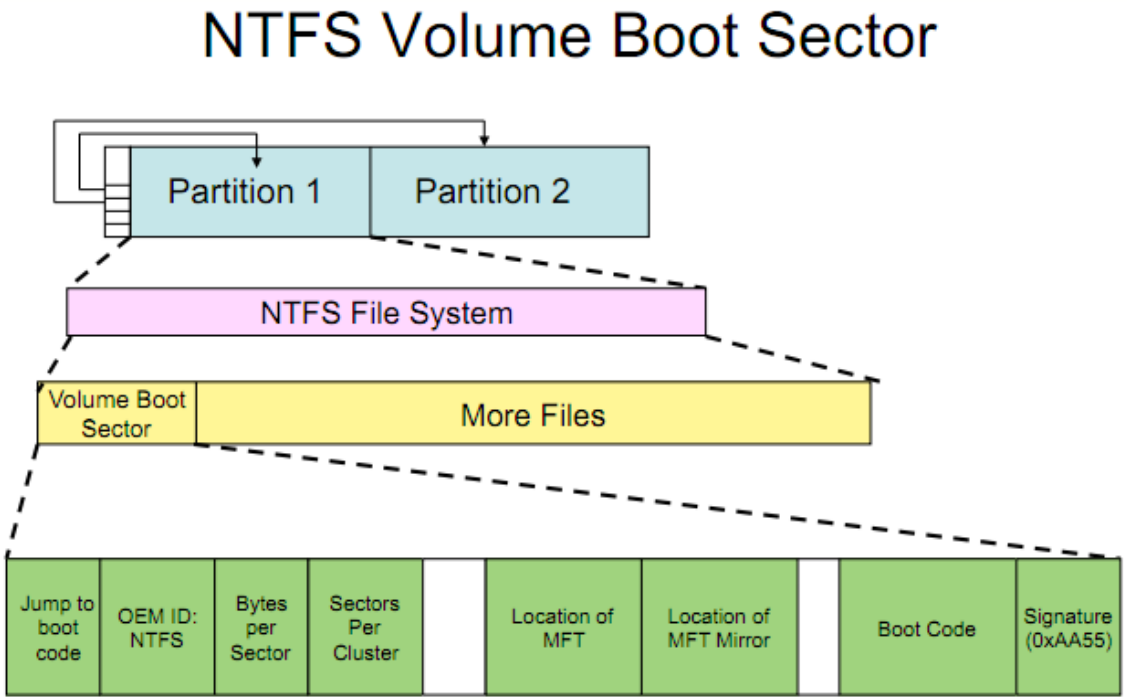
- Cluster size affects slack space volume and the likelihood of residual data recovery
- Sector alignment influences how tools interpret low-level disk structures
- Older MBR-based systems may present fragmented or multi-partition evidence, requiring careful reconstruction
- GPT-based systems are generally easier to interpret, but still rely on NTFS cluster-based allocation internally

NTFS File System Features

ROYAL HOLLOWAY UNIVERSITY OF LONDON

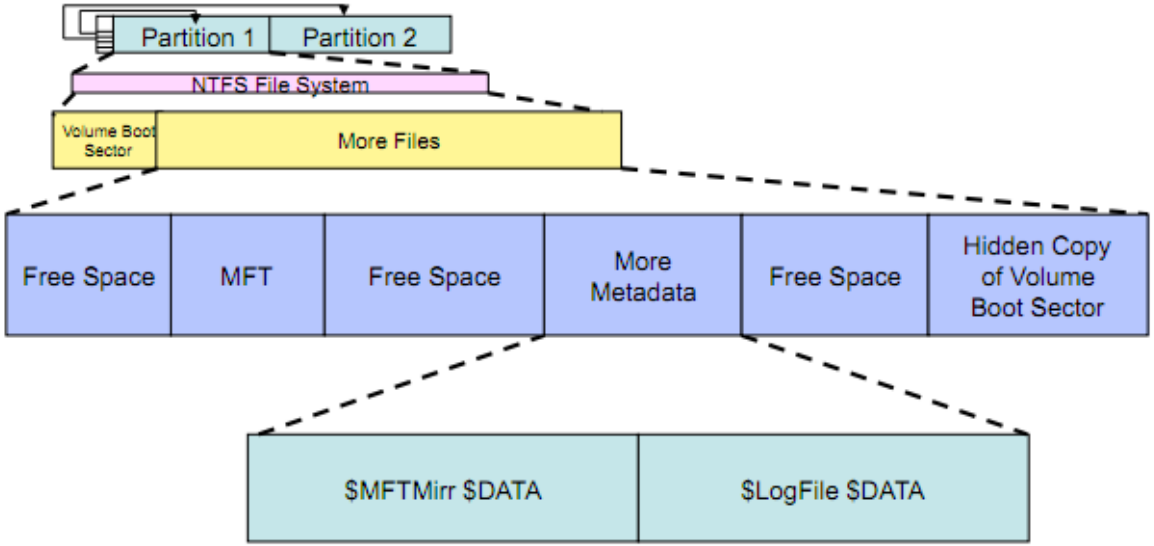
- **Data Streams Files**
  - can consist of multiple **attributes** (NTFS object attributes, which in turn are represented as **streams**.
  - Each stream can have different allocation and file sizes and can be **locked** separately
- **Indexing**
  - File attributes (including security, stored in a single stream) are indexable and do not (often) require linear searches for attributes
- **Remapping**
  - When used on a fault-tolerant file system (e.g., RAID), read failures will result in sectors (clusters) being marked "bad", and a new instance being created from another fault-tolerant copy.
  - If no "good" copy can be located, NTFS will mark the sector as bad, but cannot retrieve the data
- **Hard Links**
  - Supported for files, but not for directories
- **Symbolic Links**
  - Supported for files and directories, dynamic strings interpreted at run-time – implemented as **reparse points** that can also span volumes and arbitrary non-local file systems. Older versions of NTFS supported only **junctions**, i.e., machine-local reparse points
- **Compression and Sparse Files**
  - NTFS implements entropy (LZ77) encoding compression natively.
  - Sparse files are also supported, i.e., for large files space is not allocated immediately for file areas that are empty when marked explicitly via FSCTL_SET_SPARSE
- **Change Logs**
  - NTFS can notify applications of changes to selected areas (directories) and also explicitly supports change journals
- **Volume Quotas**
  - Both hard and soft quotas can be tracked on a per-user basis
- **Encryption**
  - EFS provides file-level encryption but cannot encrypt files needed during the boot process
- **POSIX Semantics**
  - Case-sensitive file names and traversal permissions are (were) provided in compliance to POSIX.1 / IEEE 1003.1

NTFS


NTFS Volume Boot Sector
Partition 1
Partition 2
NTFS File System
Volume Boot Sector
More Files
Jump to boot code
OEM ID: NTFS
Bytes per Sector
Sectors Per Cluster
Location of MFT
Location of MFT Mirror
Boot Code
Signature (0xAA55)

NTFS

Metadata in Center of Volume



NTFS Fundamental Files

MFT	Master File Table
	Describes all files on the volume
Cluster bitmap	A map of all the clusters on the hard drive

- Files that store file system administrative data
  - Note that they are files, unlike FAT which was a separate data structure
- Name begins with a $ and first letter is capitalised
  - $MFT
  - $LogFile

File Reference Numbers and Records

ROYAL HOLLOWAY UNIVERSITY OF LONDON

All files in an NTFS volume are uniquely identified by a 64bit value (the **File Reference**)

- This consists of a file and a sequence number
- The file number is the position of the (base) record in the MFT minus one (48 bits)
- The sequence number (16 bit) is incremented on each re-use of a MFT entry

Files are considered as **structured** entities containing attribute/value pairs

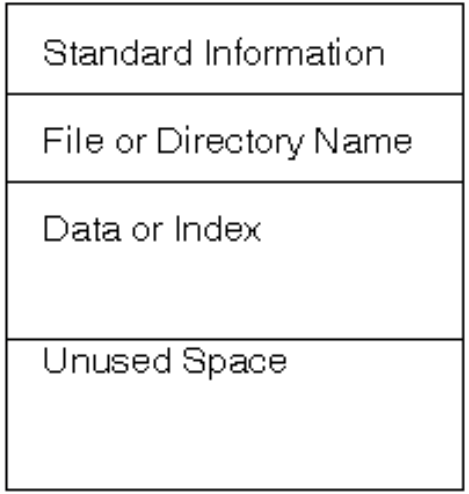
- Actual payload data is considered the **Unnamed Data Attribute**
- Each attribute is stored as a separate stream of bytes within the file

NTFS Master File Table (1)

First four entries are replicated, so that MFT can be repaired

First 16 records are reserved for metadata files, their name begins with a dollar sign ($)

- Small Files (<900B) are contained completely in the MFT entry.
- Folders contain index data.
- Small folders reside within the MFT record
- Larger folders have an index structure to other data blocks. They use a B-tree structure.
- $MFT record = 1024 bytes



NTFS Master File Table (2)

ROYAL HOLLOWAY UNIVERSITY OF LONDON

For most files a single MFT record is sufficient, but files with many attributes may require additional space

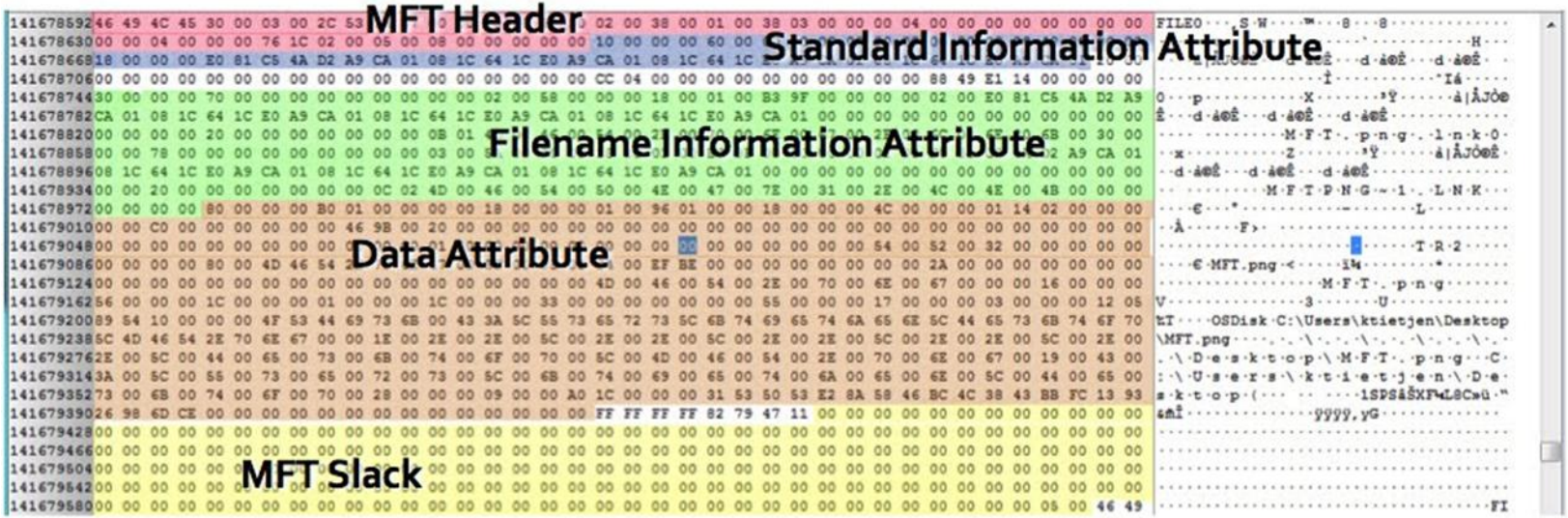
- A **base file record** then stores locations of all other MFT entries

On mounting a volume, the NTFS FSD reads the metadata structures and constructs the in-memory data structures

- The volume boot sector provides the physical (sector) address of the MFT
- The MFT is always the first entry, while the second record points to a **MFT mirror** located elsewhere on disk.
- The MFT mirror does not contain all records but only metadata records to help in case of a corrupt MFT

MFT Record Structure

- Entries are 1KB each
- Entries contain
  - File Attributes
  - Location Data



$STANDARD_INFORMATION	$FILE_NAME	$DATA
Core metadata attribute	Stores filename and directory location	Holds file content
Stores MAC timestamps	Maintains its own timestamps	Resident data: Stored inside the MFT
Updated by system processes	Multiple $FILE_NAME attributes may exist	Non-resident data: Stored elsewhere on disk
Reflects internal file system activity		

Resident and Non-Resident Attributes

Where possible, NTFS attempts to pack all file attributes in the MFT record:

- Entries within the MFT entry are called **resident** attributes
- Each attribute begins with an **Attribute Header** followed by an **Attribute Value**
- Attribute headers are always resident, values may not be
- For entries that do not fit into an MFT record, **runs** or **extents** are allocated
- Each file may have multiple runs associated with it
- Files with large numbers of attributes (or more typically, of runs, ≥ 200) may require multiple MFT entries
- Sparse files can simply be encoded by populated runs

MFT Structure: $Extend Attributes (1/2)

Name	Description
$Quota	Quota information
$ObjID	Distributed Link Tracking information
$Reparse	Back-references to reparse points
$RmMetadata	Resource Manager metadata directory
$RmMetadata\$Repair	RM repair information
$RmMetadata\$TxFLog	Transactional NTFS log directory
$RmMetadata\$TxF	TxF metadata directory

MFT Structure: $Extend Attributes (2/2)

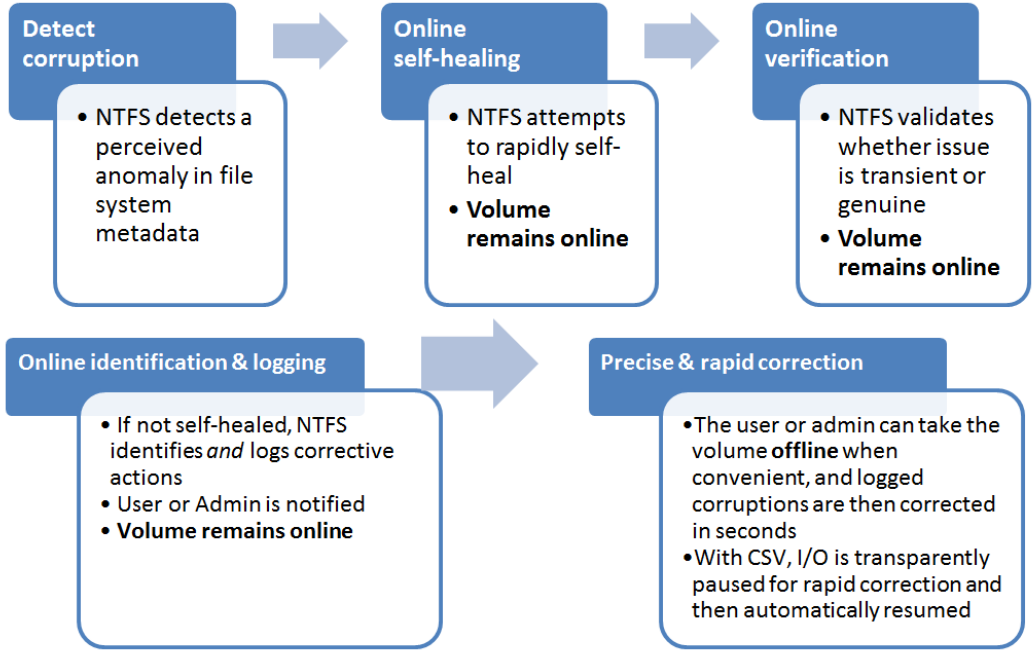
Name	Description
$RmMetadata\$TxFLog\$Tops	TxF transaction logs
$RmMetadata\$TxFLog.blf	TxF base log file

The transaction log directory contains the TxF base log file (*$TxfLog.blf*) log container files

- The number of logs depends on the size of the transaction log
- At least two files always exist, the **Kernel Transaction Manager (KTM) log stream** and the **TxF log stream**
- The transaction log directory also contains the TxF old page stream (*$Tops*)

Fault Detection and Recovery

Conventionally, the fault detection mechanisms flag inconsistencies between meta-data sets and result in a requirement to run *chkdsk*; this has been improved in Windows 8 for less severe corruption:



NTFS Deletion and Reconstruction

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Storing and Deleting a File in Windows (NTFS)

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Storing

- MFT contains one base file record for each file and directory
- MFT serves same purpose as FAT
- Cluster bitmap file maps all clusters on disk

Deleting

- When a file is deleted, data is not removed from disk
- Clusters are marked as deleted and "moved" to Recycle Bin
  - system-protected folder named **$Recycle.Bin**
- When Recycle Bin is emptied, clusters marked as fully available
- Filename in the MFT is marked with a special character that means the file has been deleted

Creating and Writing a NTFS File

ROYAL HOLLOWAY UNIVERSITY OF LONDON

The first two steps are required only when mounting a new volume and would otherwise already be cached:

1. Read volume boot sector to determine MFT location
2. Read first MFT entry, determining MFT layout
3. Allocate (reserve) MFT entry for file
4. Initialize MFT entry with *$Standard_Information*, set flag as **In Use**
5. Given file size, find free clusters in MFT $Bitmap (uses best fit algorithm), set corresponding $Bitmap fields to 1 for allocated clusters
6. Write file content to cluster, updating $Data attribute with starting address of cluster run and run length(s)
7. Read root directory, traverse index, and find matching directory (recursion as needed)
8. Read $INDEX_ROOT for directory (recurse if necessary)
9. Once correct index entry is found, create new index entry and re-sort (balance) index tree

- Each step is entered in *$LogFile* as these are taken
- INDX data structures contain references to MFT record numbers, creation and modification as well as access times and file sizes together with the file name – NTFS forensics hence normally has to rely heavily on intact INDX structures

Reconstructing NTFS Structures

ROYAL HOLLOWAY UNIVERSITY OF LONDON

- NTFS utilizes a number of more complex data structures and optimizations, making analysis somewhat more difficult besides not having a fully documented on-disk structure
- Indices (B-Trees) are used to provide sorted data structures
  - including directories and the security attribute list from NTFS 3.0 onwards
- Data structures are often more dynamic and likely to be re-used and re-allocated
- Relevant information may be retained in the MFT, log files, directory structures, the cluster bitmap, and actual attributes (including the anonymous attribute)
- Retaining short data attributes in the MFT entry makes it easier to recover short files on NTFS volumes

NTFS File Deletion Process (1/3)

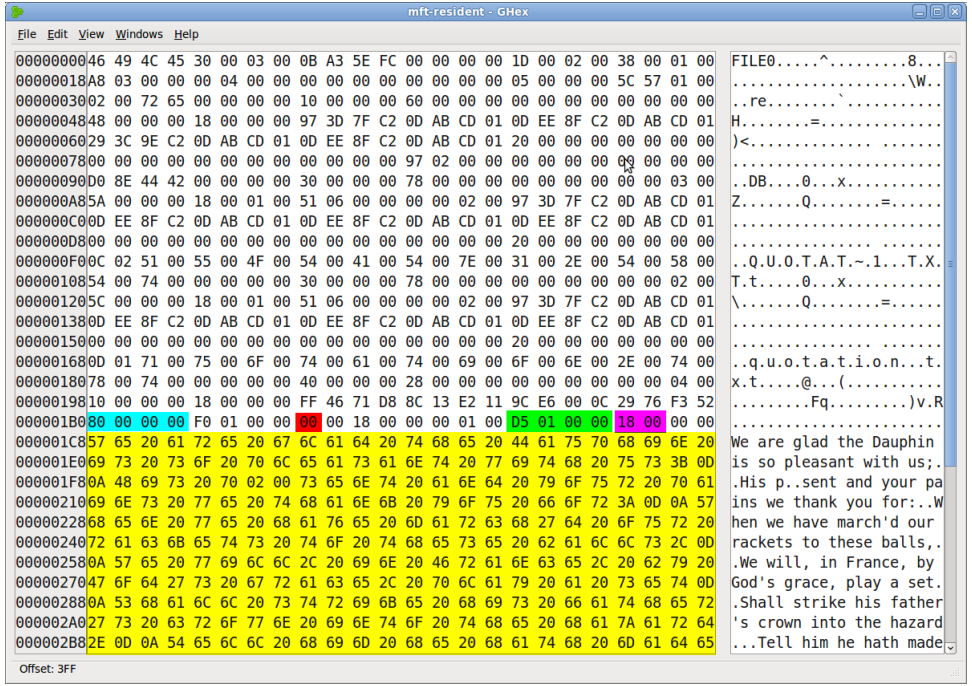
On deletion of a file in an NTFS volume (if not wholly contained in the MFT), the following steps are executed:

1. MFT is located (via boot sector, if needed)
2. First MFT entry is read (if not in memory) to determine and reconstruct MFT layout
3. Root directory is read, index traversed, and directory entry found
4. Directory hierarchy is traversed as needed
5. For the target directory, the *$Index_Root* structure is read to identify the target file
6. Target file name is removed from index, and data structure is re-balanced
7. MFT entry is un-allocated, In-Use flag is zeroed
8. Corresponding MFT *$Bitmap* entries are zeroed

All steps are entered in logs

NTFS File Deletion Process (2/3)

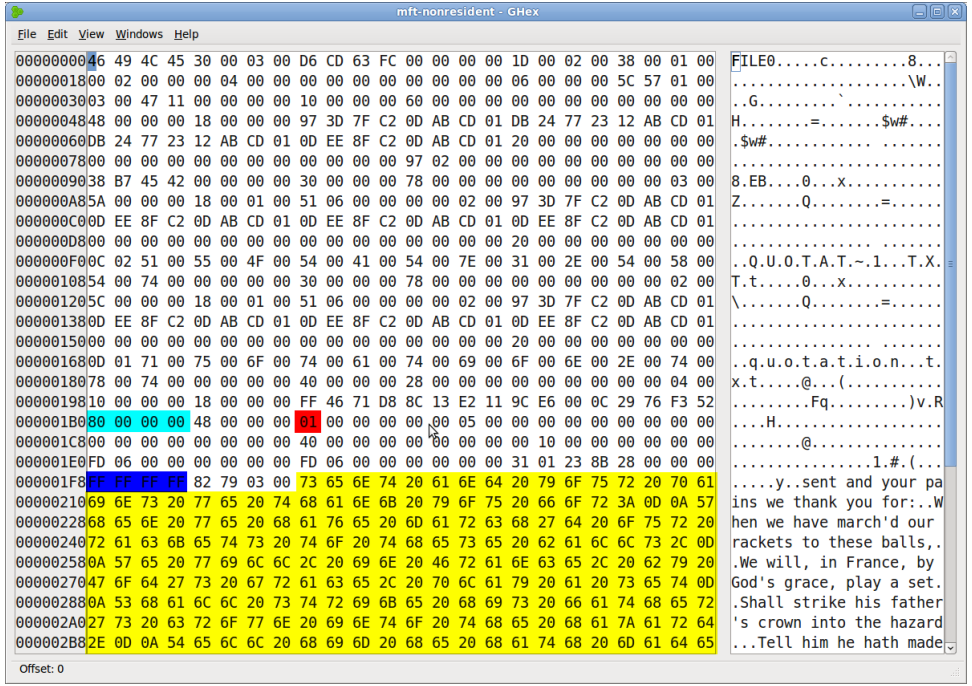
If a MFT entry of a file is no longer sufficient for resident attributes, some residue may remain even if non-resident attributes are overwritten:



Resident *$DATA* attribute

NTFS File Deletion Process (3/3)

If a MFT entry of a file is no longer sufficient for resident attributes, some residue may remain even if non-resident attributes are overwritten:



Residue in MFT despite *$DATA* now being non-resident

Deletion: Compare FAT vs NTFS

Aspect	FAT	NTFS
Deletion	Directory entry marked deleted	MFT record marked unused
Metadata persistence	Minimal	Extensive
File carving	Often effective	Depends on fragmentation
Timeline reconstruction	Weak	Stronger but complex

Anti-Forensics and Limits

- Timestamp manipulation
- Renaming and reallocation
- Secure deletion tools
- SSDs and TRIM

- Absence of evidence is not evidence of absence.

Discussion point

- Discuss the common misconceptions in File System Forensics as you understand them from today's session.
- Which of these do you think is the most dangerous assumption?

x Deleted means gone

x Timestamps show exactly what the user did

x More metadata means more certainty

x One artefact is enough

Summary

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Summary

- FAT teaches you how files are stored
- NTFS teaches you how activity is recorded
- Digital forensics lives in the gap between the two

Questions?

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Appendix

ROYAL HOLLOWAY UNIVERSITY OF LONDON

FAT Reconstruction Techniques

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Reconstruction Techniques

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Reconstruction of on-disk data depends on a number of factors including:

- Has the medium been used in some way after the incident?
- Has any deletion occurred?
- Is the use of **wiping** tools suspected?
- Is the presence of anti-forensic or malware components suspected?
- Is the file system format used known?
- Is the file system complete (e.g., not just a part of a stripe set)
- Is knowledge of file content or file format available?
- Is sought-after information replicated (whether at the file or file system level)?

Reconstructing FAT File Systems

Establishing that a volume holds a FAT file system can be done in multiple ways

- Finding the 0x55AA boot sector signature or (for FAT32) in the reserved area is often the quickest approach
- Alternatively, even for damaged file systems, the structure of a FAT can often be hypothesized

Next challenge is often to identify the first cluster

- This is easy only if the exact FAT type and cluster size is known

Finally, knowledge of the operating system version which created (or used) the file system is important in many cases

- Cluster allocation strategy may vary

Reconstructing Directory Structures

Once a directory entry has been located (this may require traversing chains of directories), the base entry must be identified

- Recall that the allocation status is set by the first byte of the entry (0x00 for a blank entry, and 0xE5 for a deleted entry)
- For long file names, multiple entries may exist
- The base directory entry will also hold the starting cluster of the file
- On creating a new directory, the OS will generally zero (wipe) the cluster, the first two entries are for "." and ".." entries

The FAT structure will then contain a chained list of entries for all data clusters

- The last block will contain an End-of-File (EOF) marker

FAT Directory Ambiguities and Orphans

Deleted (de-allocated) files will result in deletion of first byte of directory entry

- This can cause ambiguity in file names
- Long file name entries (if they remain) can be used to reconstruct original file name

More problematic: Directory deletion and re-allocation

- If a directory entry is zeroed out, files are now orphaned
- Recovery requires use of heuristics for file or directory structures
- This may sometimes find files (or fragments of files) of earlier "generations"
- Time values in entries are not always reliable, but can provide circumstantial evidence

Reconstructing Files and File Structures

Processes for reconstructing files are similar as for directories

- Where cluster information is available, simply following the chain is straightforward
- Otherwise, content-specific heuristics may be needed

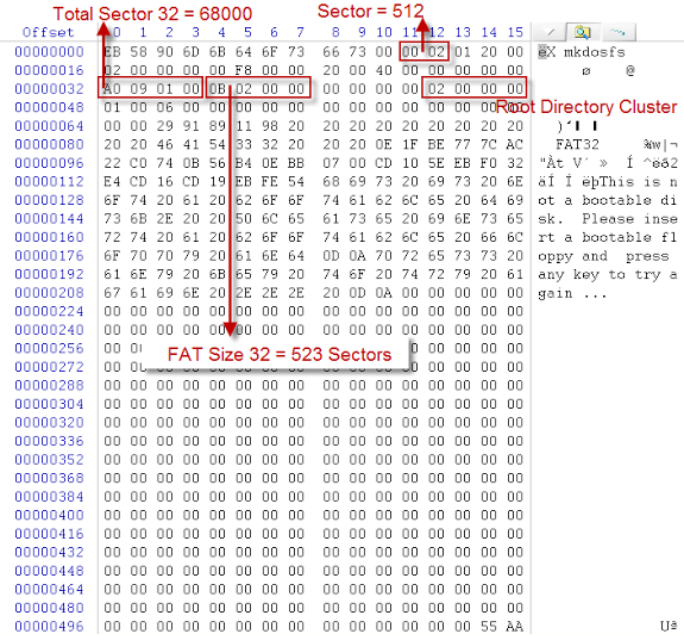
As data is allocated on a cluster basis, **on average half of the last cluster will be un-used**

- For re-used clusters, this means that data in this **slack space** may remain for some time until it is overwritten

Approach is applicable to other file systems as well, but utility may vary depending on cluster sizes, file sizes, and (re-)allocation strategies

Manual Analysis: FAT Example (1/6)

The beginning of the undamaged disk (boot sector) allows identification of file system, root directory cluster, and size of FAT table:



Manual Analysis Example (2/6)

Reserved size in FAT32 is 32 sectors (here: of 512 bytes), positioning FAT1 at offset 0x4000:

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00004000	F8	FF	FF	0F	FF	FF	FF	0F	F8	FF	FF	0F	04	00	00	00
00004010	05	00	00	00	06	00	00	00	07	00	00	00	08	00	00	00
00004020	09	00	00	00	0A	00	00	00	0B	00	00	00	0C	00	00	00
00004030	0D	00	00	00	0E	00	00	00	0F	00	00	00	10	00	00	00
00004040	11	00	00	00	12	00	00	00	13	00	00	00	14	00	00	00
00004050	15	00	00	00	16	00	00	00	17	00	00	00	18	00	00	00
00004060	19	00	00	00	1A	00	00	00	1B	00	00	00	1C	00	00	00
00004070	1D	00	00	00	1E	00	00	00	1F	00	00	00	20	00	00	00
00004080	21	00	00	00	22	00	00	00	23	00	00	00	24	00	00	00
00004090	FF	FF	FF	0F	00	00	00	00	00	00	00	00	00	00	00	00
000040A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000040B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000040C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000040D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000040E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000040F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00004100	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00004110	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00004120	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00004130	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00004140	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00004150	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00004160	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00004170	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00004180	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00004190	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000041A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000041B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000041C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000041D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000041E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000041F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Manual Analysis Example (3/6)

Here, the size of the FAT is 267776 bytes, so the backup FAT is located at an additional offset of 0x41600:

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00045600	F8	FF	FF	0F	FF	FF	FF	0F	F8	FF	FF	0F	04	00	00	00
00045610	05	00	00	00	06	00	00	00	07	00	00	00	08	00	00	00
00045620	09	00	00	00	0A	00	00	00	0B	00	00	00	0C	00	00	00
00045630	0D	00	00	00	0E	00	00	00	0F	00	00	00	10	00	00	00
00045640	11	00	00	00	12	00	00	00	13	00	00	00	14	00	00	00
00045650	15	00	00	00	16	00	00	00	17	00	00	00	18	00	00	00
00045660	19	00	00	00	1A	00	00	00	1B	00	00	00	1C	00	00	00
00045670	1D	00	00	00	1E	00	00	00	1F	00	00	00	20	00	00	00
00045680	21	00	00	00	22	00	00	00	23	00	00	00	24	00	00	00
00045690	FF	FF	FF	0F	00	00	00	00	00	00	00	00	00	00	00	00
000456A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000456B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000456C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000456D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000456E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000456F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00045700	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00045710	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00045720	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00045730	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00045740	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00045750	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00045760	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00045770	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00045780	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00045790	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000457A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000457B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000457C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000457D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000457E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000457F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

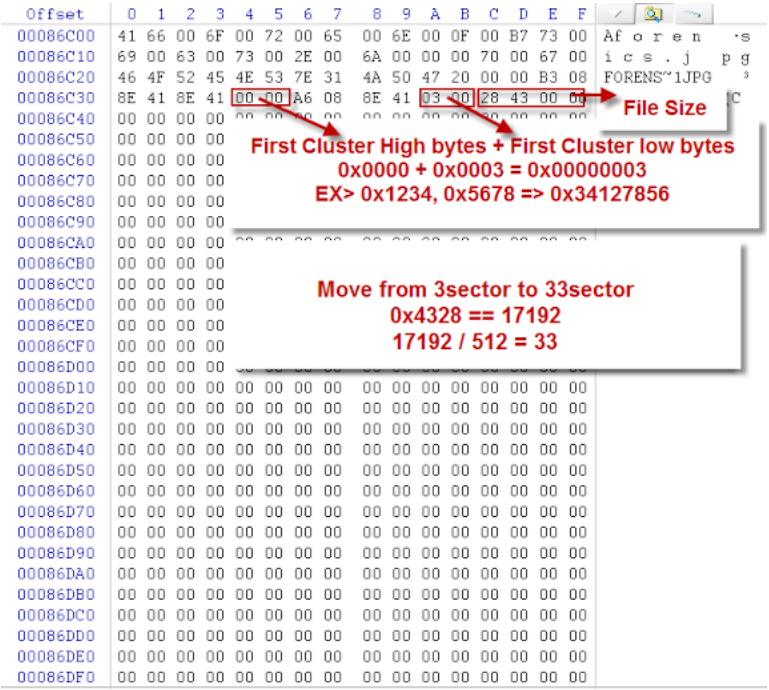
Manual Analysis Example (4/6)

This now allows identification of directory offsets and entries

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
00086C00	41	66	00	6F	00	72	00	65	00	6E	00	0F	00	B7	73	00	Af o r e n ·s
00086C10	69	00	63	00	73	00	2E	00	6A	00	00	00	70	00	67	00	i c s . j p g
00086C20	46	4F	52	45	4E	53	7E	31	4A	50	47	20	00	00	B3	08	FORENS~1JPG ³
00086C30	8E	41	8E	41	00	00	A6	08	8E	41	03	00	28	43	00	00	▮A▮A ¦ ▮A (C
00086C40	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086C50	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086C60	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086C70	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086C80	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086C90	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086CA0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086CB0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086CC0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086CD0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086CE0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086CF0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086D00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086D10	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086D20	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086D30	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086D40	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086D50	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086D60	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086D70	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086D80	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086D90	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086DA0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086DB0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086DC0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086DD0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086DE0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00086DF0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Manual Analysis Example (5/6)

The file (conveniently located in the root directory) contains 33 sectors:



Manual Analysis Example (6/6)

This has now allowed localization of the first cluster, and we see the “magic number” for a JPEG image file:

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
00086E00	FF	D8	FF	E0	00	10	4A	46	49	46	00	01	00	01	00	60	ÿØÿà JFIF `
00086E10	00	60	00	00	FF	FE	00	1F	4C	45	41	44	20	54	65	63	` ÿþ LEAD Tec
00086E20	68	6E	6F	6C	6F	67	69	65	73	20	49	6E	63	2E	20	56	hnologies Inc. V
00086E30	31	2E	30	31	00	FF	DB	00	84	00	05	05	05	08	05	08	1.01 ÿÛ ▌
00086E40	0C	07	07	0C	0C	09	09	09	0C	0D	0C	0C	0C	0C	0D	0D	
00086E50	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	
00086E60	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	
00086E70	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	01	05	08	08	0A	07	
00086E80	0A	0C	07	07	0C	0D	0C	0A	0C	0D	0D	0D	0D	0D	0D	0D	
00086E90	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	
00086EA0	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	
00086EB0	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	0D	FF	C4	01	A2	00	ÿÄ ¢
00086EC0	00	01	05	01	01	01	01	01	01	00	00	00	00	00	00	00	
00086ED0	00	01	02	03	04	05	06	07	08	09	0A	0B	01	00	03	01	
00086EE0	01	01	01	01	01	01	01	01	00	00	00	00	00	00	01	02	
00086EF0	03	04	05	06	07	08	09	0A	0B	10	00	02	01	03	03	02	
00086F00	04	03	05	05	04	04	00	00	01	7D	01	02	03	00	04	11	}
00086F10	05	12	21	31	41	06	13	51	61	07	22	71	14	32	81	91	!1A Qa "q 2 ‘
00086F20	A1	08	23	42	B1	C1	15	52	D1	F0	24	33	62	72	82	09	¡ #B±Á RÑð$3br▌
00086F30	0A	16	17	18	19	1A	25	26	27	28	29	2A	34	35	36	37	%&'()*4567
00086F40	38	39	3A	43	44	45	46	47	48	49	4A	53	54	55	56	57	89:CDEFGHIJSTUVW
00086F50	58	59	5A	63	64	65	66	67	68	69	6A	73	74	75	76	77	XYZcdefghijstuvw
00086F60	78	79	7A	83	84	85	86	87	88	89	8A	92	93	94	95	96	xyz▌▌▌▌▌▌▌▌´▌▌▌▌
00086F70	97	98	99	9A	A2	A3	A4	A5	A6	A7	A8	A9	AA	B2	B3	B4	▌▌▌▌¢£¤¥¦§¨©ª²³´
00086F80	B5	B6	B7	B8	B9	BA	C2	C3	C4	C5	C6	C7	C8	C9	CA	D2	µ¶·¸¹ºÂÃÄÅÆÇÈÉÊÒ
00086F90	D3	D4	D5	D6	D7	D8	D9	DA	E1	E2	E3	E4	E5	E6	E7	E8	ÓÔÕÖ×ØÙÚáâãäåæçè
00086FA0	E9	EA	F1	F2	F3	F4	F5	F6	F7	F8	F9	FA	11	00	02	01	éêñòóôõö÷øùú
00086FB0	02	04	04	03	04	07	05	04	04	00	01	02	77	00	01	02	w
00086FC0	03	11	04	05	21	31	06	12	41	51	07	61	71	13	22	32	!1 AQ aq "2
00086FD0	81	08	14	42	91	A1	B1	C1	09	23	33	52	F0	15	62	72	B´¡±Á #3Rð br
00086FE0	D1	0A	16	24	34	E1	25	F1	17	18	19	1A	26	27	28	29	Ñ $4á%ñ &'()
00086FF0	2A	35	36	37	38	39	3A	43	44	45	46	47	48	49	4A	53	*56789:CDEFGHIJS

Deleted Files (1/4)

Recall that deleted files (using basic operating system functions) in FAT file systems will simply replace the first byte in the directory entries with oxE5 (highlighted in yellow), here of a folder called *My Pictures*:

```
00137920  E5 4D 00 79 00 20 00 50  00 69 00 0F 00 BF 63 00  åM.y. .P.i...¿c.
00137936  74 00 75 00 72 00 65 00  73 00 00 00 00 00 FF FF  t.u.r.e.s.....ÿÿ
00137952  E5 59 50 49 43 54 7E 31  20 20 20 10 00 26 D5 7E  åYPICT~1   ..&Õ~
00137968  3B 30 3B 30 00 00 D6 7E  3B 30 46 08 00 08 00 00  ;0;0..Ö~;0F.....
00137984  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00138000  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
```

Here, the base directory entry is highlighted in red, the extended (long file name) entry in blue; file attributes are highlighted in cyan

- Time stamps, first and last cluster of the directory can also be recovered

Deleted Files (2/4)

This entry still allows recovery of the file name (except for the first character) and the first cluster (2118); together with basic volume information this allows calculation of the correct offset (here: 0x444E00):

00444DF0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ
00444E00	2E	20	20	20	20	20	20	20	20	20	20	10	00	26	D5	7E	. ..&Õ~
00444E10	3B	30	3B	30	00	00	D6	7E	3B	30	46	08	00	00	00	00	;0;0..Ö~;0F.....
00444E20	2E	2E	20	20	20	20	20	20	20	20	20	10	00	26	D5	7E	.. ..&Õ~
00444E30	3B	30	3B	30	00	00	D6	7E	3B	30	00	00	00	00	00	00	;0;0..Ö~;0.....
00444E40	E5	6B	00	69	00	74	00	74	00	69	00	0F	00	EF	65	00	åk.i.t.t.i...ïe.
00444E50	20	00	70	00	6F	00	72	00	6E	00	00	00	00	00	FF	FF	.p.o.r.n.....ÿÿ
00444E60	E5	49	54	54	49	45	7E	31	20	20	20	10	00	56	DD	7E	åITTIE~1 ..VÝ~
00444E70	3B	30	3B	30	00	00	E0	7E	3B	30	47	08	00	08	00	00	;0;0..à~;0G.....
00444E80	E5	61	00	64	00	75	00	6C	00	74	00	0F	00	60	20	00	åa.d.u.l.t...` .
00444E90	70	00	6F	00	72	00	6E	00	00	00	00	00	FF	FF	FF	FF	p.o.r.n.....ÿÿÿÿ
00444EA0	E5	44	55	4C	54	50	7E	31	20	20	20	10	00	C4	E7	7E	åDULTP~1 ..Äç~
00444EB0	3B	30	3B	30	00	00	E8	7E	3B	30	48	08	00	08	00	00	;0;0..è~;0H.....
00444EC0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	.....

Self (.) and parent entries (..) are first, then there are folders called *kittiporn* and *adultporn…*

Deleted Files (3/4)

Traversing the first directory, the folder *kittiporn* was located at cluster 0x0847 (reversed byte order owing to endianness on x86 systems):

```
004455F0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00445600  2E 20 20 20 20 20 20 20  20 20 20 10 00 56 DD 7E  .             ..VÝ~
00445610  3B 30 3B 30 00 00 E0 7E  3B 30 47 08 00 00 00 00  ;0;0..à~;0G.....
00445620  2E 2E 20 20 20 20 20 20  20 20 20 10 00 56 DD 7E  ..            ..VÝ~
00445630  3B 30 3B 30 00 00 E0 7E  3B 30 46 08 00 00 00 00  ;0;0..à~;0F.....
00445640  E5 2D 00 74 00 00 00 FF  FF FF FF 0F 00 B3 FF FF  å-.t...ÿÿÿÿ..³ÿÿ
00445650  FF FF FF FF FF FF FF FF  FF FF 00 00 FF FF FF FF  ÿÿÿÿÿÿÿÿÿÿ..ÿÿÿÿ
00445660  E5 43 00 41 00 35 00 35  00 2E 00 0F 00 B3 33 00  åC.A.5.5.....³3.
00445670  30 00 34 00 34 00 33 00  38 00 00 00 2D 00 31 00  0.4.4.3.8...-.1.
00445680  E5 41 35 35 7E 31 20 20  33 30 34 20 00 5D 08 7F  åA55~1  304 .].▮
00445690  3B 30 3B 30 00 00 09 7F  3B 30 00 00 00 00 00 00  ;0;0...▮;0......
004456A0  E5 2D 00 74 00 00 00 FF  FF FF FF 0F 00 B3 FF FF  å-.t...ÿÿÿÿ..³ÿÿ
004456B0  FF FF FF FF FF FF FF FF  FF FF 00 00 FF FF FF FF  ÿÿÿÿÿÿÿÿÿÿ..ÿÿÿÿ
004456C0  E5 43 00 41 00 35 00 35  00 2E 00 0F 00 B3 33 00  åC.A.5.5.....³3.
004456D0  30 00 34 00 34 00 33 00  38 00 00 00 2D 00 31 00  0.4.4.3.8...-.1.
004456E0  E5 41 35 35 7E 31 20 20  33 30 34 20 00 5D 08 7F  åA55~1  304 .].▮
004456F0  3B 30 3B 30 00 00 01 7F  3B 30 49 08 C1 07 00 00  ;0;0...▮;0I.Á...
00445700  E5 31 00 2D 00 74 00 00  00 FF FF 0F 00 3B FF FF  å1.-.t...ÿÿ..;ÿÿ
00445710  FF FF FF FF FF FF FF FF  FF FF 00 00 FF FF FF FF  ÿÿÿÿÿÿÿÿÿÿ..ÿÿÿÿ
00445720  E5 43 00 41 00 35 00 35  00 2E 00 0F 00 3B 31 00  åC.A.5.5.....;1.
00445730  37 00 39 00 30 00 37 00  37 00 00 00 33 00 2D 00  7.9.0.7.7...3.-.
```

Deleted Files (4/4)

The actual file is then found from cluster 0x0849 onwards:

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	Access
00446600	FF	D8	FF	E0	00	10	4A	46	49	46	00	01	01	00	00	01	ÿØÿà..JFIF.....
00446610	00	01	00	00	FF	DB	00	43	00	08	06	06	07	06	05	08	....ÿÛ.C.....
00446620	07	07	07	09	09	08	0A	0C	14	0D	0C	0B	0B	0C	19	12	.....
00446630	13	0F	14	1D	1A	1F	1E	1D	1A	1C	1C	20	24	2E	27	20	..... $.'
00446640	22	2C	23	1C	1C	28	37	29	2C	30	31	34	34	34	1F	27	",#..(7),01444.'
00446650	39	3D	38	32	3C	2E	33	34	32	FF	DB	00	43	01	09	09	9=82<.342ÿÛ.C...
00446660	09	0C	0B	0C	18	0D	0D	18	32	21	1C	21	32	32	32	32	.....2!.!2222
00446670	32	32	32	32	32	32	32	32	32	32	32	32	32	32	32	32	2222222222222222
00446680	32	32	32	32	32	32	32	32	32	32	32	32	32	32	32	32	2222222222222222
00446690	32	32	32	32	32	32	32	32	32	32	32	32	32	32	FF	C0	22222222222222ÿÀ
004466A0	00	11	08	00	32	00	48	03	01	22	00	02	11	01	03	11	....2.H..".....
004466B0	01	FF	C4	00	1F	00	00	01	05	01	01	01	01	01	01	00	.ÿÄ.....
004466C0	00	00	00	00	00	00	00	01	02	03	04	05	06	07	08	09	.....
004466D0	0A	0B	FF	C4	00	B5	10	00	02	01	03	03	02	04	03	05	..ÿÄ.µ.....
004466E0	05	04	04	00	00	01	7D	01	02	03	00	04	11	05	12	21	.....}.....!
004466F0	31	41	06	13	51	61	07	22	71	14	32	81	91	A1	08	23	1A..Qa."q.2▮´ì.#

… a JPEG image with a cat picture

MFT Structure (1/2)

Seq.	Name	Description
0	$MFT	The MFT itself
1	$MFTMirr	Partial MFT mirror (protects against single-sector failure only)
2	$LogFile	Log file (file and volume transaction steps, actual log file has up to 4MB)
3	$Volume	Volume information, including label and version
4	$AttrDef	Table of attribute names, numbers, and descriptions
5	$\	The root folder
6	$Bitmap	Bitmap of volume with cluster usage
7	$Boot	Boot sector including the BIOS Parameter Block (if volume is bootable)

MFT Structure (2/2)

Seq.	Name	Description
8	$BadClus	Bad cluster aggregate
9	$Secure	Security descriptors for all files
10	$Upcase	Conversion table for matching Unicode uppercase characters
11	$Extend	Extension file location for file system extensions
12-23		(reserved for extensions)
	⋮	
33		(Beginning of user files and directories – exact location varies)

The exact location of the extension records depends on the file system and kernel version

NTFS File Attributes (1/3)

Attribute Type	Description
$VOLUME_INFORMATION	Special entry for $Volume file only (volume version)
$VOLUME_NAME	Special entry for $Volume file only (volume label)
$STANDARD_INFORMATION	Read-only, archive, etc. attributes, timestamps
$FILE_NAME	Unicode file name. Multiple name attributes may exist (e.g., 8.3 names, hard links)

NTFS File Attributes (2/3)

Attribute Type	Description
$SECURITY_DESCRIPTOR	Backwards compatibility only (NTFS V3.0 25 stores security attributes in $Secure metadata)
$DATA	Payload data
$INDEX_ROOT	Hierarchical directory structure data
$INDEX_ALLOCATION	(directory structure)
$INDEX_BITMAP	(directory structure)
$ATTRIBUTE_LIST	If more than one MFT entry is needed: List of attributes and reference to further MFT records

NTFS File Attributes (3/3)

Attribute Type	Description
$OBJECT_ID	64byte identifier, lowest 128 bits are unique to volume
$REPARSE_POINT	Content of mount points and junctions
$EA	Extended attributes (OS/2 backwards compatibility only)
$EA_INFORMATION	(as above)
$LOGGED_UTILITY_STREAM	EFS data for performing encryption and decryption